

Iteration Revisited

Tristan Brindle



ITERATION BASICS

```
int arr[] = {1, 2, 3, 4, 5};  
size_t sz = 5;  
  
for (size_t i = 0; i < sz; i++) {  
    do_something(arr[i]);  
}
```

ITERATION BASICS

```
for (size_t i = 0; i < sz; i++) {  
    do_something(arr[i]);  
}
```

ITERATION BASICS

```
for (int* p = arr; p  $\neq$  arr + sz; ++p) {  
    do_something(*p);  
}
```

ITERATION BASICS

```
for (int* p = std::begin(arr); p != std::end(arr); p++) {  
    do_something(*p);  
}
```

ITERATION BASICS

```
for (auto iter = std::begin(arr);  
    iter != std::end(arr);  
    ++iter) {  
    do_something(*iter);  
}
```

STL ITERATORS

- ▶ STL iterators are a *generalisation* of array pointers
- ▶ Pros:
 - ▶ Powerful
 - ▶ Low overhead
 - ▶ “Natural” syntax
- ▶ Cons:
 - ▶ Iterators are **just as dangerous** as raw pointers

WHAT DO I MEAN BY SAFETY?

 **Cppcon 2022**
The C++ Conference
September 12th-16th







Dave Abrahams

Values: Safety, Regularity, Independence, and the Future of Programming

Reference semantics | Safety

Definitions
A safe operation cannot cause undefined behavior.
A safe language has only safe operations.

Technical debt

Spooky action

Incidental algorithms

Visibly broken invariants

Race conditions

Surprise mutation

Unspecifiable mutation



 Adobe



Video Sponsorship Provided By:

12:37 / 1:00:45 • Safety >

Value Semantics: Safety, Independence, Projection, & Future of Programming - Dave Abrahams CppCon 22



CppCon
129K subscribers



Subscribed

318



Share

Clip

Save



**A SAFE OPERATION CANNOT CAUSE
UNDEFINED BEHAVIOUR.**

Dave Abrahams

ITERATORS AND UB

```
auto iter = std::end(rng);  
do_something(*iter);
```

- ▶ *Dereferencing* a past-the-end iterator is UB

ITERATORS AND UB

```
auto iter = std::end(rng);  
++iter;
```

- ▶ *Incrementing* a past-the-end iterator is UB
 - ▶ So is decrementing a `begin()` iterator
 - ▶ So are out-of-bounds random-access jumps

ITERATORS AND UB

```
auto min_iter(const std::vector<int>& vec)
{
    return std::min_element(vec.begin(), vec.end());
}
```

```
auto iter = min_iter(get_vector());
std::cout << "Minimum is " << *iter << std::endl;
```

- ▶ **Dangling iterators** can occur easily
 - ▶ **Any** operation on a dangling iterator is UB

ITERATORS AND UB

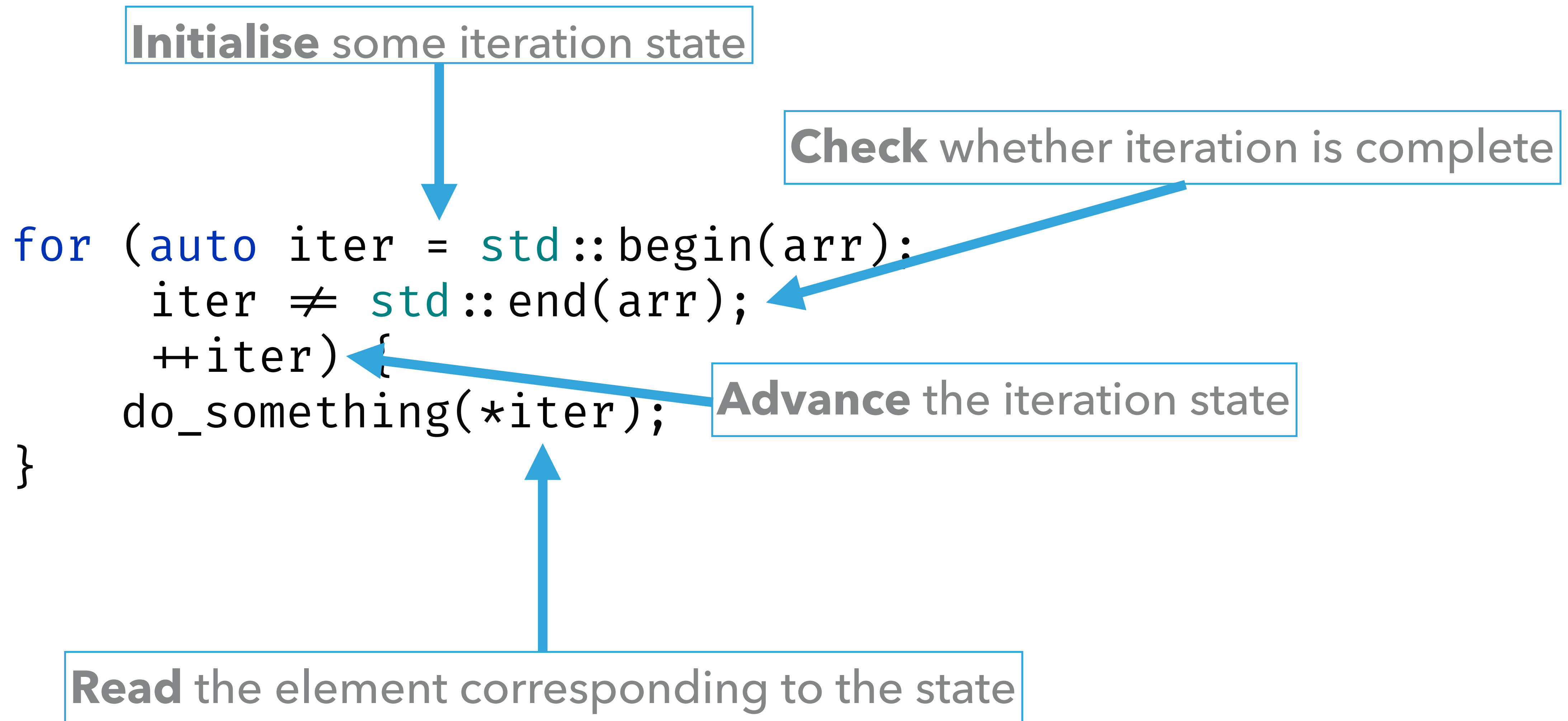
```
auto iter = vec.begin();  
  
vec.push_back(99);  
  
while (iter != vec.end()) {  
    ...  
}
```

- ▶ Iterators can become dangling in non-obvious ways
 - ▶ All part of the C++ experience!

ITERATORS AND UB

- ▶ “A safe operation cannot cause undefined behaviour”
- ▶ Most iterator operations can cause UB
- ▶ => STL iterators are a ***fundamentally unsafe*** abstraction

ITERATION FUNDAMENTALS



ITERATION FUNDAMENTALS

- ▶ **Initialise** some iteration state
- ▶ **Check** whether iteration is complete
- ▶ **Read** the element corresponding to the state
- ▶ **Advance** the iteration state

Rust iterators:

`next() -> optional<T>`

ITERATION FUNDAMENTALS

Iterators and Ranges: Comparing C++ to D, Rust, and Others



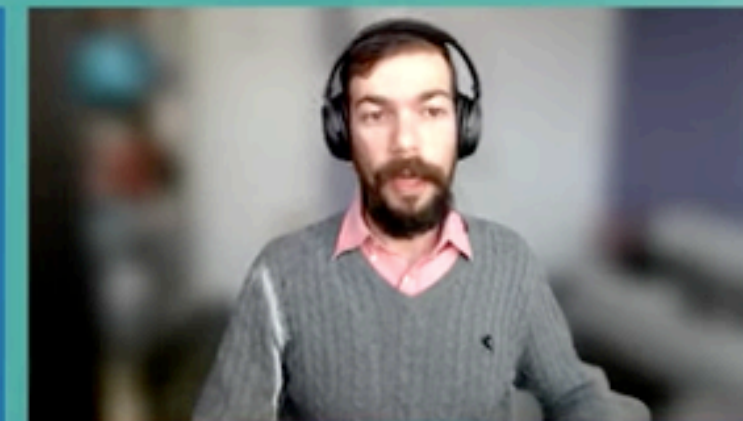
Implementing map in C++ (transform)

```
template <input_range V, copy_constructible F>
    requires view<V> &&
        regular_invocable<F&, range_reference_t<V>>
class map_view {
    struct Iterator;
    struct Sentinel;

    V base_;
    F fun_;

public:
    map_view(V, F);

    auto begin() -> Iterator;
    auto end()   -> Sentinel;
};
```



Barry Revzin



CODE RECKONS

Science to the CORE

Keynote: Iterators and Ranges: Comparing C++ to D, Rust, and Others - Barry Revzin - CPPP 2021



CPPP Conference
816 subscribers

Subscribe

47



Share

Clip

Save



ITERATION WITH INDICES

```
std::vector<int> vec{1, 2, 3, 4, 5};  
  
for (size_t i = 0; i < vec.size(); i++) {  
    do_something(vec.at(i));  
}
```

ITERATION WITH INDICES

Initialise some iteration state

Check whether iteration is complete

```
for (size_t i = 0; i < vec.size(); i++) {  
    do_something(vec.at(i));  
}
```

Advance the iteration state

Read the element corresponding to the state

ITERATION WITH INDICES

```
auto idx = std::size(vec);  
do_something(vec.at(idx));
```

- ▶ Reading via a past-the-end index is bounds checked
 - ▶ No UB!

ITERATION WITH INDICES

```
auto idx = std::size(vec);  
++idx;
```

- ▶ *Incrementing a past-the-end index is fine*
 - ▶ (Assuming unsigned indices or checked operations on signed indices)

ITERATION WITH INDICES

```
size_t min_idx(const std::vector<int>& vec) {  
    ...  
}  
  
auto idx = min_idx(get_vector());  
// std::cout << "Minimum is " << ??? << std::endl;
```

- ▶ “Dangling indices” are safe **by design**

ITERATION WITH INDICES

```
size_t idx = 0;  
vec.push_back(99);  
while (idx < vec.size()) {  
    ...  
}
```

- ▶ “Index invalidation” problems are no longer UB
 - ▶ Caught by bounds check at next attempted read

ITERATION WITH INDICES

- ▶ Index-based iteration **dramatically reduces** the potential for UB
- ▶ ...but is still **just as powerful**
- ▶ The Big Idea: generalise iteration by *index*, rather than by *pointer*

FLUX

- ▶ **Flux** (github.com/tcbrindle/flux) is a C++20 library exploring these ideas
- ▶ Aims:
 - ▶ Much improved safety by default vs STL iterators
 - ▶ Improved ease-of-use and fewer “gotchas”
 - ▶ Equal or better runtime performance vs C++20 ranges
 - ▶ Compatibility with existing STL code

ITERATION OPERATIONS WITH FLUX

- ▶ `flux::first(seq)` returns a *cursor*, which represents a sequence position
 - ▶ For contiguous sequences, this is just an integer index
- ▶ `flux::is_last(seq, cur)` returns whether the cursor is past-the-end
- ▶ `flux::read_at(seq, cur)` accesses the element at the cursor position
- ▶ `flux::inc(seq, cur)` increments the cursor

FLUX CONCEPTS

- ▶ A type which provides these four operations models the **sequence** concept
- ▶ Sequences are single-pass by default
- ▶ *Multipass* sequences are those which allow simultaneous iteration by independent cursors
- ▶ A sequence whose cursor is a *regular type* is assumed to be multipass
 - ▶ Regular = default constructible, copyable, equality comparable
 - ▶ Sequences can opt out of being multipass if necessary

FLUX CONCEPTS

- ▶ *Bidirectional sequences* are multipass sequences which additionally allow their cursors to be *decremented*:
 - ▶ `flux::dec(seq, cur)`
- ▶ *Random-access sequences* provide additional operations:
 - ▶ `flux::inc(seq, cur, offset)`
 - ▶ `flux::distance(seq, from_cur, to_cur)`
 - ▶ Cursors are `std::totally_ordered`
- ▶ *Contiguous sequences* are random-access, backed by an in-memory array

FLUX CONCEPTS

- ▶ *Bounded sequences* allow accessing a past-the-end cursor in constant time
 - ▶ `flux::last(seq)`
- ▶ *Sized sequences* allow accessing the number of elements in constant time
 - ▶ `flux::size(seq)`
- ▶ *Infinite sequences* guarantee that `is_last()` always returns `false`

FLUX SEQUENCE ALGORITHMS

► As of June 2023

all
any
compare
contains
count
count_eq
count_if
contains
ends_with
equal
fill
find
find_if
find_if_not

fold
fold_first
for_each
for_each_while
inplace_reverse
max
min
minmax
none
output_to
product
search
sort
starts_with

sum
swap_elements
to
write_to
zip_find_if
zip_fold
zip_for_each
zip_for_each_while

FLUX PIPELINES

- ▶ Like C++20 Ranges, Flux provides a variety of *sequence adaptors* performing lazy evaluation
- ▶ Typically these are composed into pipelines
- ▶ A pipeline begins with some sequence (the *source*), passes through zero or more *adaptors*, and then finishes with a call to an *algorithm*

FLUX PIPELINE EXAMPLE

```
std::vector<int> vec = get_vector();

int max_even_sq = flux::ref(vec)           // iterate over a reference to vec
                  .filter(flux::pred::even) // keep only even numbers
                  .map([](int i) { return i * i; }) // square each int
                  .max()                       // find the maximum value
                  .value_or(0);                // handle empty sequences
```

FLUX SEQUENCE ADAPTORS

► As of June 2023

adjacent
adjacent_map
cache_last
cartesian_product
cartesian_product_with
chain
chunk
chunk_by
cursors
cycle
drop
drop_while
filter
flatten

map
pairwise
pairwise_map
prescan
read_only
reverse
scan
scan_first
set_difference
set_intersection
set_symmetric_difference
set_union
slide
stride
split

take
take_while
unchecked
zip

IMPLEMENTING THE SEQUENCE PROTOCOL

```
template <typename T, std::size_t N>
struct my_array {
    T data[N];
};

template <typename T, std::size_t N>
struct flux::sequence_traits<my_array<T, N>> {
    using self_t = my_array<T, N>;

    static size_t first(const self_t&) { return 0; }
    static bool is_last(const self_t&, size_t cur) { return cur ≥ N; }
    static void inc(const self_t&, size_t& cur) { ++cur; }
    static decltype(auto) read_at(auto& self, size_t cur) {
        flux::bounds_check(cur < N);
        return self.data[cur];
    }
};

static_assert(flux::multipass_sequence<my_array<int, 10>>);
```

IMPLEMENTING THE SEQUENCE PROTOCOL

```
template <typename T, std::size_t N>
struct my_array {
    T data[N];

    struct flux_sequence_traits {
        static size_t first(const my_array&) { return 0; }
        static bool is_last(const my_array&, size_t cur) { return cur ≥ N; }
        static void inc(const my_array&, size_t& cur) { ++cur; }
        static decltype(auto) read_at(auto& self, size_t cur)
        {
            flux::bounds_check(cur < N);
            return self.data[cur];
        }
    };
};

static_assert(flux::multipass_sequence<my_array<int, 10>>);
```

IMPLEMENTING THE SEQUENCE PROTOCOL

- ▶ Considerably easier to implement vs STL ranges
 - ▶ Even more so for higher sequence/range categories
- ▶ No `iterator/const_iterator` confusion: the same cursor type is used for `const` and `non-const` access

RANGES COMPATIBILITY

- ▶ Every C++20 `contiguous_range` is automatically a `flux::contiguous_sequence`
 - ▶ Contiguous => "cheaply indexable"
 - ▶ Covers vector, array, string, string_view, span...
- ▶ Other ranges can be wrapped in a sequence implementation using `flux::from_range(rng)`, with reduced safety guarantees
 - ▶ No worse than using ranges directly

RANGES COMPATIBILITY

- ▶ Conversely, every `flux::sequence` provides STL-compatible iterators
 - ▶ So we can use them with C++20 algorithms and range adaptors
 - ▶ ...and range-for loops

```
template <flux::sequence Seq>
struct sequence_iterator {
    Seq* seq;
    flux::cursor_t<Seq> cur;

    sequence_iterator& operator++() {
        flux::inc(*seq, cur);
        return *this;
    }

    // etc...
};
```



PERFORMANCE

- ▶ Compilers are **incredibly good** at eliding bounds checks
- ▶ Many (most?) new languages use bounds checking universally
- ▶ => Optimisers can recognise those patterns in very many cases
- ▶ For example: <https://flux.godbolt.org/z/v35dsd7sx>

Private

flux.godbolt.org

COMPILER EXPLORER

Add... More Templates

Watch C++ Weekly to learn new C++ features

Sponsors

intel

CONAN

Solid Sands

Share Policies 41 Other

C++ source #1

Diff Viewer x86-64 gcc 13.1 vs x86-64 gcc 13.1

C++

Left: x86-64 gcc 13.1 -std=c++2b ...

Assembly

Right: x86-64 gcc 13.1 -std=c++2b ...

Assembly

```
int count_evens(std::vector<int> const& vec)
{
    int count = 0;
    for (int i : vec) {
        if (i % 2 == 0) {
            ++count;
        }
    }
    return count;
}
```

```
13 jbe .L12
14
15
16
17
18
19 .L4:
20 movdqu xmm0, XMMWORD PTR [rax]
21 add rax, 16
22 pand xmm0, xmm3
23 pcmpeqd xmm0, xmm2
24 psubd xmm1, xmm0
25 cmp rax, rdx
26 jne .L4
27 movdqa xmm0, xmm1
28 mov rdx, rsi
29 psrldq xmm0, 8
30 and rdx, -4
31 paddb xmm1, xmm0
32 movdqa xmm0, xmm1
33 psrldq xmm0, 4
34 paddb xmm1, xmm0
35 movd eax, xmm1
36 test sil, 3
37 je .L18
38 .L3:
39 mov edi, DWORD PTR [rcx+rdx*4]
40 lea r8, [0+rdx*4]
41 and edi, 1
42 cmp edi, 1
43 lea rdi, [rdx+1]
44 adc eax, 0
45 cmp rsi, rdi
```

x86-64 gcc 13.1 (Editor #1)

x86-64 gcc 13.1

-std=c++2b -O3 -DNDEBUG -DUSE_FLUX

x86-64 gcc 13.1 (Editor #1)

x86-64 gcc 13.1

-std=c++2b -O3 -DNDEBUG

Private

flux.godbolt.org

COMPILER EXPLORER

Add... More Templates

Watch C++ Weekly to learn new C++ features

Sponsors intel CONAN Solid Sands

Share Policies 42 Other

C++ source #1

#ifndef USE_FLUX

int count_evens(std::vector<int> const& vec)

{

int count = 0;

for (int i : vec) {

if (i % 2 == 0) {

++count;

}

}

}

Diff Viewer x86-64 gcc 13.1 vs x86-64 gcc 13.1

Left: x86-64 gcc 13.1 -std=c++2b ...

Assembly

Right: x86-64 gcc 13.1 -std=c++2b ...

Assembly

13 jbe .L12

14 mov rcx, rsi

15 pxor xmm1, xmm1

16 movdqa xmm3, XMMWORD PTR .LC0[rip]

17 shr rcx, 2

18 movdqa xmm2, xmm1

19 sal rcx, 4

20 add rcx, rcx

21 .L4:

22 movdqu xmm0, XMMWORD PTR [rax]

23

24

25

26

27

28

29

30

31

32

33

34

35

36

37

38 .L3:

39

40

41

42

43

44

45

46

47

12 pxor xmm1, xmm1

13 movdqa xmm3, XMMWORD PTR .LC0[rip]

14+ mov rax, rcx

15+ shr rdx, 2

16 movdqa xmm2, xmm1

17+ sal rdx, 4

18+ add rdx, rcx

19 .L4:

20 movdqu xmm0, XMMWORD PTR [rax]

21 add rax, 16

22 pand xmm0, xmm3

23 pcmpeqd xmm0, xmm2

24 psubd xmm1, xmm0

25+ cmp rax, rdx

26 jne .L4

27 movdqa xmm0, xmm1

28+ mov rdx, rsi

29 psrldq xmm0, 8

30+ and rdx, -4

31 paddd xmm1, xmm0

32 movdqa xmm0, xmm1

33 psrldq xmm0, 4

34 paddd xmm1, xmm0

35 movd eax, xmm1

36+ test sil, 3

37 je .L18

38 .L3:

39+ mov edi, DWORD PTR [rcx+rdx*4]

40+ lea r8, [0+rdx*4]

41+ and edi, 1

42+ cmp edi, 1

43+ lea rdi, [rdx+1]

44 adc eax, 0

45+ cmp rsi, rdi

int count_evens_flux(std::vector<int> const& vec)

{

int count = 0;

FLUX_FOR(int i, vec) {

if (i % 2 == 0) {

++count;

}

}

return count;

}

x86-64 gcc 13.1 (Editor #1)

x86-64 gcc 13.1

-std=c++2b -O3 -DNDEBUG -DUSE_FLUX

x86-64 gcc 13.1 (Editor #1)

x86-64 gcc 13.1

-std=c++2b -O3 -DNDEBUG

Private

flux.godbolt.org

COMPILER EXPLORER

Add... More Templates

Watch C++ Weekly to learn new C++ features

Sponsors intel CONAN Solid Sands

Share Policies 43 Other

C++ source #1

C++

```
1 #ifndef USE_FLUX
2
3 int count_evens(std::vector<int> const& vec)
4 {
5     int count = 0;
6     for (int i : vec) {
7         if (i % 2 == 0) {
8             ++count;
9         }
10    }
11    return count;
12 }
13
14 #else
15
16 int count_evens_flux(std::vector<int> const& vec)
17 {
18     int count = 0;
19     FLUX_FOR(int i, vec) {
20         if (i % 2 == 0) {
21             ++count;
22         }
23     }
24     return count;
25 }
26
27 #endif
28
29
```

Diff Viewer x86-64 gcc 13.1 vs x86-64 gcc 13.1

Left: x86-64 gcc 13.1 -std=c++2b ... Assembly

Right: x86-64 gcc 13.1 -std=c++2b ... Assembly

13 jbe .L12

14 mov rcx, rsi

15 pxor xmm1, xmm1

16 movdqa xmm3, XMMWORD PTR .LC0[rip]

17 shr rcx, 2

18 movdqa xmm2, xmm1

19 sal rcx, 4

20 add rcx, rcx

21 .L4:

22 movdqu xmm0, XMMWORD PTR [rax]

23 add rax, 16

24 pand xmm0, xmm3

25 pcmpeqd xmm0, xmm2

26 psubd xmm1, xmm0

27 cmp rax, rcx

28 jne .L4

29 movdqa xmm0, xmm1

30 mov rcx, rsi

31 psrldq xmm0, 8

32 and rcx, -4

33 and esi, 3

34 paddb xmm1, xmm0

35 lea rdx, [rdx+rcx*4]

36 movdqa xmm0, xmm1

37 psrldq xmm0, 4

38 paddb xmm1, xmm0

39 movd eax, xmm1

40 je .L18

41 .L3:

42 mov ecx, DWORD PTR [rdx]

43 and ecx, 1

44 cmp ecx, 1

45 lea rcx, [rdx+4]

46 adc eax, 0

47 cmp rdi, rcx

12 pxor xmm1, xmm1

13 movdqa xmm3, XMMWORD PTR .LC0[rip]

14+ mov rax, rcx

15+ shr rdx, 2

16 movdqa xmm2, xmm1

17+ sal rdx, 4

18+ add rdx, rcx

19 .L4:

20 movdqu xmm0, XMMWORD PTR [rax]

21 add rax, 16

22 pand xmm0, xmm3

23 pcmpeqd xmm0, xmm2

24 psubd xmm1, xmm0

25+ cmp rax, rdx

26 jne .L4

27 movdqa xmm0, xmm1

28+ mov rdx, rsi

29 psrldq xmm0, 8

30+ and rdx, -4

31 paddb xmm1, xmm0

32 movdqa xmm0, xmm1

33 psrldq xmm0, 4

34 paddb xmm1, xmm0

35 movd eax, xmm1

36+ test sil, 3

37 je .L18

38 .L3:

39+ mov edi, DWORD PTR [rcx+rdx*4]

40+ lea r8, [0+rdx*4]

41+ and edi, 1

42+ cmp edi, 1

43+ lea rdi, [rdx+1]

44 adc eax, 0

45+ cmp rsi, rdi

x86-64 gcc 13.1 (Editor #1)

x86-64 gcc 13.1

✓

-std=c++2b -O3 -DNDEBUG -DUSE_FLUX

x86-64 gcc 13.1 (Editor #1)

x86-64 gcc 13.1

✓

-std=c++2b -O3 -DNDEBUG

PERFORMANCE

- ▶ This implementation is using the `FLUX_FOR( )` macro to force iteration using Flux primitives (`first, is_last, read_at, inc`)
- ▶ Idiomatic use of Flux would be to use a named algorithm instead, i.e.
`flux::count_if( )`

Private

flux.godbolt.org

COMPILER EXPLORER

Add... More Templates

Watch C++ Weekly to learn new C++ features

Sponsors intel CONAN Solid Sands

Share Policies 45 Other

C++ source #1

C++

```
1 #ifndef USE_FLUX
2
3 int count_evens(std::vector<int> const& vec)
4 {
5     int count = 0;
6     for (int i : vec) {
7         if (i % 2 == 0) {
8             ++count;
9         }
10    }
11    return count;
12 }
13
14 #else
15
16 int count_evens_flux(std::vector<int> const& vec)
17 {
18     return flux::count_if(vec, flux::pred::even);
19 }
20
21 #endif
22
23 
```

Diff Viewer x86-64 clang 16.0.0 vs x86-64 clang 16.0.0

Left: x86-64 clang 16.0.0 -std=c++2b -O3 -DNDEBUG -DUSE_FLUX

Assembly

Right: x86-64 clang 16.0.0 -std=c++2b -O3 -DNDEBUG -DUSE_FLUX

Assembly

1 .LCPI0_0:

2 .long 1 # 0x1

3 .long 1 # 0x1

4 .long 1 # 0x1

5 .long 1 # 0x1

6 count_evens_flux(std::vector<int, std::allocator<int> > const&): # @c

7 mov r8, qword ptr [rdi]

8 mov rcx, qword ptr [rdi + 8]

9 cmp r8, rcx

10 je .LBB0_1

11 mov rsi, rcx

12 sub rsi, r8

13 add rsi, -4

14 xor eax, eax

15 cmp rsi, 28

16 jae .LBB0_4

17 mov rdx, r8

18 jmp .LBB0_7

19 .LBB0_1:

20 xor eax, eax

21 ret

22 .LBB0_4:

23 shr rsi, 2

24 inc rsi

25 mov rdi, rsi

26 and rdi, -8

27 lea rdx, [r8 + 4*rdi]

28 pxor xmm0, xmm0

29 xor eax, eax

30 movdqa xmm2, xmmword ptr [rip + .LCPI0_0] # xmm2 = [1,1,1,1

31 pxor xmm1, xmm1

32 .LBB0_5: # =>This Inner Loop Header: 1

33 movdqu xmm3, xmmword ptr [r8 + 4*rax]

34 movdqu xmm4, xmmword ptr [r8 + 4*rax + 16]

35 pandn xmm3, xmm2

36 paddb xmm0, xmm3

37 pandn xmm4, xmm2

38 paddb xmm1, xmm4

1 .LCPI0_0:

2 .long 1 # 0x1

3 .long 1 # 0x1

4 .long 1 # 0x1

5 .long 1 # 0x1

6 count_evens(std::vector<int, std::allocator<int> > const&): # @c

7 mov r8, qword ptr [rdi]

8 mov rcx, qword ptr [rdi + 8]

9 cmp r8, rcx

10 je .LBB0_1

11 mov rsi, rcx

12 sub rsi, r8

13 add rsi, -4

14 xor eax, eax

15 cmp rsi, 28

16 jae .LBB0_4

17 mov rdx, r8

18 jmp .LBB0_7

19 .LBB0_1:

20 xor eax, eax

21 ret

22 .LBB0_4:

23 shr rsi, 2

24 inc rsi

25 mov rdi, rsi

26 and rdi, -8

27 lea rdx, [r8 + 4*rdi]

28 pxor xmm0, xmm0

29 xor eax, eax

30 movdqa xmm2, xmmword ptr [rip + .LCPI0_0] # xmm2 = [1,1,1,1

31 pxor xmm1, xmm1

32 .LBB0_5: # =>This Inner Loop Header: 1

33 movdqu xmm3, xmmword ptr [r8 + 4*rax]

34 movdqu xmm4, xmmword ptr [r8 + 4*rax + 16]

35 pandn xmm3, xmm2

36 paddb xmm0, xmm3

37 pandn xmm4, xmm2

38 paddb xmm1, xmm4

x86-64 clang 16.0.0 (Editor #1)

x86-64 clang 16.0.0

-std=c++2b -O3 -DNDEBUG -DUSE_FLUX

x86-64 clang 16.0.0 (Editor #1)

6-64 clang 16.0.0

-std=c++2b -O3 -DNDEBUG

INTERNAL ITERATION

- ▶ The Flux sequence protocol has an extra, optional customisation point:
`for_each_while(predicate)`
- ▶ This instructs the sequence to perform *internal iteration* until the predicate returns `false`
- ▶ Many sequence adaptors can perform internal iteration more efficiently than by a check/read/advance loop
- ▶ Many algorithms can be written in terms for `for_each_while` (e.g. `fold()`, `find_if()`, `min()`)
- ▶ => Flux pipelines can in many cases generate more efficient code than equivalent C++20 Ranges pipelines

```

1
2     #include <cstdint>
3     #include <stdint.h>
4     #include <vector>
5
6     #ifdef USE_FLUX
7
8     #include "https://raw.githubusercontent.com/tcbrindle/flux/main/single\_i
9
10    int sum_sq_even_flux(std::vector<int> const& vec)
11    {
12        return flux::ref(vec)
13            .filter(flux::pred::even)

```

```

14
15
16 int sum_sq_even_ranges(std::vector<int> const& vec)
17
18 {
19
20     auto view = vec
21
22         | std::views::filter([](int i) { return i % 2 == 0; })
23
24         | std::views::transform([](int i) { return i * i; });
25
26     return std::accumulate(view.begin(), view.end(), 0);
27
28 }
29
30
31
32
33

```

A screenshot of the clang compiler selection dropdown menu. The menu is open, showing a list of compilers. The selected compiler is 'x86_64 clang 16.0.0', which is highlighted with a green checkmark. The dropdown is part of a larger window titled 'x86-64 clang 16.0.0 (Editor #1)'. The dropdown list also shows the compiler version 'x86-64 clang 16.0.0' and the compiler flags '-std=c++20 -O3 -DNDEBUG -m'.

```

30      vpxor    xmm3, xmm3, xmm3
31      vpxor    xmm4, xmm4, xmm4
32      LBB0_5:                                     # =>This Inner Loop Header: Depth=1
33      vmovdqu  ymm5, ymmword ptr [r8 + 4*rax]
34      vmovdqu  ymm6, ymmword ptr [r8 + 4*rax + 32]
35      vmovdqu  ymm7, ymmword ptr [r8 + 4*rax + 64]
36      vmovdqu  ymm8, ymmword ptr [r8 + 4*rax + 96]
37      vptestnmd k1, ymm5, ymm1
38      vptestnmd k2, ymm6, ymm1

```

```
:vector<int> const& vec)

r([](int i) { return i % 2 == 0; })
form([](int i) { return i * i; });
view.begin(), view.end(), 0);
```

```

1  sum_sq_even_ranges(std::vector<int, std::allocator<int> > const&): # @sum_
2      mov     rcx, qword ptr [rdi]
3      mov     rdx, qword ptr [rdi + 8]
4      cmp     rcx, rdx
5      je      .LBB0_4
6  .LBB0_1:                                     # =>This Inner Loop Header: Depth=
7      test    byte ptr [rcx], 1
8      je      .LBB0_4
9      add     rcx, 4
10     cmp     rcx, rdx
11     jne     .LBB0_1
12     jmp     .LBB0_3
13 .LBB0_4:
14     cmp     rcx, rdx
15     je      .LBB0_3
16     mov     esi, dword ptr [rcx]
17     xor     eax, eax
18 .LBB0_6:                                     # =>This Loop Header: Depth=1
19     mov     edi, eax
20     imul    esi, esi
21     mov     eax, esi
22     add     eax, edi
23 .LBB0_7:                                     #   Parent Loop BB0_6 Depth=1
24     add     rcx, 4
25     cmp     rcx, rdx
26     je      .LBB0_10
27     mov     esi, dword ptr [rcx]
28     test    sil, 1
29     jne     .LBB0_7
30     cmp     rcx, rdx
31     jne     .LBB0_6
32 .LBB0_10:
33     ret
34 .LBB0_3:
35     xor     eax, eax
36     ret

```

```
1
2 #include <cstdint>

int sum_sq_even_flux(std::vector<int> const& vec)
{
    return flux::ref(vec)
        .filter(flux::pred::even)
        .map([](int i) { return i * i; })
        .sum();
}
```

```
25 {
26     auto view = vec
27     | std::views::filter([](int i) { return i % 2 == 0; });
28     | std::views::transform([](int i) { return i * i; });
29     return std::accumulate(view.begin(), view.end(), 0);
30 }
31
32 #endif
33
```

```

48      vpaddd    ymm4, {k4}, ymm4, ymm8
49      add      rax, 32
50      cmp      r9, rax
51      jne      .LBB0_5
52      vpaddd    ymm0, ymm2, ymm0
53      vpaddd    ymm1, ymm4, ymm3
54      vpaddd    ymm0, ymm1, ymm0
55      vextracti128    xmm1, ymm0, 1
56      vpaddd    xmm0, xmm0, xmm1
57      vpshufd   xmm1, xmm0, 238                # xmm1 = xmm0[2,3,2,3]
58      vpaddd    xmm0, xmm0, xmm1
59      vpshufd   xmm1, xmm0, 85                 # xmm1 = xmm0[1,1,1,1]
60      vpaddd    xmm0, xmm0, xmm1
61      vmovd     eax, xmm0
62      cmp      rdi, r9
63      je       .LBB0_8
64  LBB0_7:                                     # =>This Inner Loop Header: Depth=1
65      mov      edi, dword ptr [rsi]
66      mov      r8d, edi
67      imul     r8d, edi
68      test     dil, 1
69      cmovne   r8d, edx
70      add      eax, r8d
71      add      rsi, 4
72      cmp      rsi, rcx
73      jne      .LBB0_7
74  LBB0_8:

```

```

1  sum_sq_even_ranges(std::vector<int, std::allocator<int> > const&): # @sum_
2      mov     rcx, qword ptr [rdi]
3      mov     rdx, qword ptr [rdi + 8]
4      cmp     rcx, rdx
5      je      .LBB0_4
6  .LBB0_1:                                     # =>This Inner Loop Header: Depth=
7      test    byte ptr [rcx], 1
8      je      .LBB0_4
9      add     rcx, 4
10     cmp     rcx, rdx
11     jne     .LBB0_1
12     jmp     .LBB0_3
13 .LBB0_4:
14     cmp     rcx, rdx
15     je      .LBB0_3
16     mov     esi, dword ptr [rcx]
17     xor     eax, eax
18 .LBB0_6:                                     # =>This Loop Header: Depth=1
19     mov     edi, eax
20     imul    esi, esi
21     mov     eax, esi
22     add     eax, edi
23 .LBB0_7:                                     #   Parent Loop BB0_6 Depth=1
24     add     rcx, 4
25     cmp     rcx, rdx
26     je      .LBB0_10
27     mov     esi, dword ptr [rcx]
28     test    sil, 1
29     jne     .LBB0_7
30     cmp     rcx, rdx
31     jne     .LBB0_6
32 .LBB0_10:
33     ret
34 .LBB0_3:
35     xor     eax, eax
36     ret

```



Sponsors



Share ▼

Policies

Other ▼

□ ×

□ ×

□ ×

x86-64 clang 16.0.0    -std=c++20 -O3 -DNDEBUG -R 

```

1  sum_sq_even_ranges(std::vector<int, std::allocator<int> > const&): # @sum
2      mov     rcx, qword ptr [rdi]
3      mov     rdx, qword ptr [rdi + 8]
4      cmp     rcx, rdx
5      je      .LBB0_4
6  .LBB0_1:                                     # =>This Inner Loop Header: Depth=
7      test    byte ptr [rcx], 1
8      je      .LBB0_4
9      add     rcx, 4
10     cmp     rcx, rdx
11     jne     .LBB0_1
12     jmp     .LBB0_3
13 .LBB0_4:
14     cmp     rcx, rdx
15     je      .LBB0_3
16     mov     esi, dword ptr [rcx]
17     xor     eax, eax
18 .LBB0_6:                                     # =>This Loop Header: Depth=1
19     mov     edi, eax
20     imul    esi, esi
21     mov     eax, esi
22     add     eax, edi
23 .LBB0_7:                                     #   Parent Loop BB0_6 Depth=1
24     add     rcx, 4
25     cmp     rcx, rdx
26     je      .LBB0_10
27     mov     esi, dword ptr [rcx]
28     test    sil, 1
29     jne     .LBB0_7
30     cmp     rcx, rdx
31     jne     .LBB0_6
32 .LBB0_10:
33     ret
34 .LBB0_3:
35     xor     eax, eax
36     ret

```

AVOIDING BOUNDS CHECKING

- ▶ Implementations of `for_each_while()` typically don't need internal bounds-checking
- ▶ In most other cases, the compiler can optimise away bounds checks
- ▶ But what about corner cases?

AVOIDING BOUNDS CHECKING

- ▶ If bounds checking does lead to unacceptable overhead, Flux provides the `read_at_unchecked()` function which performs element access without additional checks
- ▶ This can be used ***selectively***, only where truly needed
 - ▶ Always measure!
- ▶ Think of it as the equivalent of `unsafe` blocks in other languages

CONCLUSION

- ▶ Flux is a modern C++20 library for *collection-orientated programming*
- ▶ It can be used everywhere C++20 ranges can be used, and offers
 - ▶ Much improved safety by default
 - ▶ An easy-to-use API for both producers and consumers
 - ▶ Compatibility with existing STL code
 - ▶ In many cases, better performance than equivalent ranges code

THANK YOU!

- ▶ Flux: github.com/tcbrindle/flux
- ▶ Docs: tristanbrindle.com/flux
- ▶ Twitter: @tristanbrindle

EXPLICIT REFERENCING IN FLUX

- ▶ C++20 range adaptors ("views") take an implicit reference when passed a non-view, lvalue range
- ▶ This can lead to surprising behaviour and "dangling views" in some cases

```
auto filter_evens(std::vector<int> const& vec)
{
    return std::views::filter(vec, [](int i) { return i % 2 == 0; });
}
```

```
auto evens = filter_evens({1, 2, 3, 4, 5}); // uh oh
```

EXPLICIT REFERENCING IN FLUX

- ▶ In Flux, potentially “long-lived” references must be explicitly passed to sequence adaptors
- ▶ Makes dangling much more obvious in code

```
auto filter_evens(std::vector<int> const& vec)
{
    return flux::filter(vec, [](int i) { return i % 2 == 0; }); // Compile error!
}
```

EXPLICIT REFERENCING IN FLUX

- ▶ In Flux, potentially “long-lived” references must be explicitly passed to sequence adaptors
- ▶ Makes dangling much more obvious in code

```
auto filter_evens(std::vector<int> const& vec)
{
    // Now compiles, but more obviously problematic
    return flux::filter(flux::ref(vec), [](int i) { return i % 2 == 0; });
}
```